MODULE 23 · DOMAIN 4 - ASSEMBLING AND DEPLOYING APPLICATIONS

Packaging a chain: pyfunc, signatures, Unity Catalog

Wrapping pre-processing, the chain and post-processing into one model, and registering it so it can be served.

SECTION 4 of 6 EXAM WEIGHT 22% MODULE 23 of 40

admin@sthithapragna.com

A packaged chain is a sandwich. Pre-processing and post-processing are the bread. Package the whole sandwich, not just the filling.

Where pre- and post-processing happen

01
Request
Raw input from any caller.

›

02
Pre-process
Validate, clean, extract fields, mask data.

›

03
Chain
Retrieve, build prompt, call the model.

›

04
Post-process
Parse, format, add citations, redact.

›

05
Response
The same shape for every caller.

ON THE EXAM

Anything every caller should get the same way belongs inside the packaged model, not in each caller.

admin@sthithapragna.com

Anatomy of a custom pyfunc

PIECE	WHAT GOES THERE
`mlflow.pyfunc.PythonModel`	The base class you subclass
`load_context(context)`	Runs once at load: clients, config, artifacts
`predict(context, model_input, params)`	Runs per request: pre, chain, post
`artifacts`	Files the model needs, such as prompts or lookup tables
`Models from code`	Alternative: the model defined in its own Python file

ON THE EXAM

Expensive setup belongs in load_context, which runs once. predict runs on every request, so keep it lean.

Inside the model, or in each caller?

IN EACH CALLER

Everyone bakes their own bread

- Every app reimplements it
- Behaviour drifts between apps
- Fixes must be made everywhere
- Hard to audit

INSIDE THE PYFUNC

One sandwich for everyone

- Written once, in predict
- Identical for every caller
- Fixed once, redeployed
- Versioned with the model

THE TIEBREAKER

When several callers need the same cleaning, masking or parsing, put it in the model's predict method.

The basic elements of a rag application

ELEMENT	WHY IT IS NEEDED
Model flavour	How MLflow saves and loads it: a built-in flavour, or pyfunc
Embedding model	Embeds queries the same way as the documents
Retriever	The index and endpoint that supply context
Dependencies	Pinned libraries, so serving can rebuild the environment
Input example	A sample request, for testing and documentation
Model signature	The input and output schema, checked at inference

ON THE EXAM

When a question lists these elements, the missing one is usually the answer. Serving needs all six.

A built-in flavour, or pyfunc?

BUILT-IN FLAVOUR

The object is standard

- mlflow.langchain, mlflow.openai and others
- MLflow knows how to save it
- Least code
- Behaviour is the library's

CUSTOM PYFUNC

You need your own logic

- Custom pre- and post-processing
- Several components glued together
- Objects no flavour supports
- Full control of predict

THE TIEBREAKER

A standard LangChain chain can use the LangChain flavour. Custom logic around it means pyfunc.

Describing what goes in and comes out

- **infer_signature**(an input, its output) builds the schema from real data.
- Models in Unity Catalog **require a signature**.
- With a signature, **inputs are checked** at inference, and mismatches raise an error.
- An **input example** documents the request and lets serving test the model.
- Without a signature: no input checks, and AI Functions need a schema passed in.

> ON THE EXAM
>
> A registration error about a missing signature has one fix: log the model again with a signature.

Making the environment rebuildable

- List libraries in **pip_requirements**, pinned to versions.
- A missing library means the **serving container fails** to build or start.
- Local helper modules go in **code_paths**, so they travel with the model.
- Declare **resources**, such as endpoints and indexes, for authentication. Module 25.
- Unpinned versions work today and break on the next rebuild.

> ON THE EXAM
>
> An error such as ModuleNotFoundError at serving time is a dependency problem. Fix pip_requirements, then re-log.

From a logged model to a governed version

01
Registry
databricks-uc, the default in MLflow 3.

›

02
Log
With signature, input example, requirements.

›

03
Register
As catalog.schema.model: a new version.

›

04
Alias
Point champion at the chosen version.

›

05
Serve
By alias or version number.

ON THE EXAM

Register with a three-level name. Unity Catalog permissions then govern who can see, use and deploy it.

Stages, or aliases?

WORKSPACE REGISTRY

Stages

- Staging and Production
- A fixed set of names
- One stage per version
- Not used in Unity Catalog

UNITY CATALOG

Aliases and tags

- Named pointers, such as champion
- Move a pointer to promote or roll back
- Load by models:/name@alias
- Tags for labels

THE TIEBREAKER

In Unity Catalog, stages are replaced by aliases for calling models and tags for labelling them.

The pieces of a simple LangChain chain

PIECE	ROLE
Retriever	Fetches chunks for the question
Formatter	Joins chunk text into one context string
Prompt template	Slots question and context into the prompt
Chat model	ChatDatabricks over a serving endpoint
Output parser	Turns the message into a plain string, or JSON

ON THE EXAM

Chained in order with the pipe operator: retrieve and format, then prompt, then model, then parser.

Harbourline packages its policy assistant

01
Pre
Mask card numbers, extract policy number.

02
Chain
Filtered retrieval, prompt, model.

03
Post
JSON answer with clause citations.

04
Log
pyfunc, signature, pinned requirements.

05
Register
harbourline.assist.policy_qa, alias champion.

ON THE EXAM

Three internal apps now call one model and get identical masking, citations and format.

> An engineer logs a custom pyfunc chain and tries to register it as main.rag.policy_qa in Unity Catalog.
>
> Registration fails with an error saying the model has no signature.

What should the engineer do?

A Register it in the workspace registry under a stage instead

B Grant the team MANAGE on the catalog, then retry the step

C Log it again with a signature inferred from sample data

D Add the chain's libraries to pip_requirements and retry

◆ PAUSE AND DECIDE

What should the engineer do?

A Workspace registry and stages

Loses Unity Catalog governance, and stages are legacy.

B Grant MANAGE and retry

The error is about the model, not permissions.

C Re-log with a signature

Unity Catalog requires one. infer_signature builds it.

D Fix pip_requirements

Dependencies matter at serving, not for this error.

THE SIGNAL

No signature, and registering to Unity Catalog. Models in Unity Catalog require a signature.

> Three internal apps call the same chain endpoint. Each app masks personal data and parses the JSON output with its own code.
>
> Behaviour differs between the apps, and a masking bug had to be fixed three times.

What is the best change?

A Move the masking and parsing into the pyfunc's predict method

B Publish a shared document describing the parsing rules

C Ask each app team to pin the same version of a parser library

D Add a guardrail that rejects every response that is not JSON

◆ PAUSE AND DECIDE

What is the best change?

A Into predict

Written once, identical for all, fixed once.

B Shared document

Three implementations still drift apart.

C Same parser version

Masking still differs, and code is still triplicated.

D Guardrail rejecting non-JSON

Blocks bad output. Does not unify masking.

THE SIGNAL

Same processing, three implementations, three fixes. Put it inside the packaged model once.

> A pyfunc chain works in the engineer's notebook. When deployed to a serving endpoint, the container fails to start.
>
> The logs show ModuleNotFoundError for a text-splitting library the chain imports.

What should the engineer fix?

A Increase the endpoint's workload size and deploy it again

B Change the model signature to include the missing package

C Move the champion alias back to the previous model version

D List the library in pip_requirements and log the model again

◆ PAUSE AND DECIDE

What should the engineer fix?

A Bigger workload

More resources for an environment still missing the library.

B Signature

Describes inputs and outputs, not packages.

C Roll back the alias

Restores service, but leaves the new version broken.

D pip_requirements, re-log

Serving rebuilds the environment from the requirements.

THE SIGNAL

Works in the notebook, fails in serving, ModuleNotFoundError. The notebook had the library; the requirements did not.

> A team wants every app to load whichever model version is currently approved for production.
>
> They also want to roll back instantly, without changing any app's code or configuration.

What should they use?

A. Move each approved version to the Production stage

B. A champion alias, loaded as models:/name@champion

C. The latest version number, set in each app's config

D. A brand-new model name registered for every release

◆ PAUSE AND DECIDE

What should they use?

A Production stage
Stages are not used in Unity Catalog.

B champion alias
Apps load the alias. Moving it promotes or rolls back.

C Version number in config
Every change means editing every app.

D New name per release
Every release means changing every app.

THE SIGNAL

Load the current approved version, and roll back without touching apps. That is exactly what an alias is.

What to carry forward

- Package the **whole sandwich**: pre-processing, chain and post-processing in predict.
- Setup in **load_context**, once. Per-request work in **predict**.
- Six elements: flavour, embedding model, retriever, dependencies, input example, **signature**.
- Unity Catalog: three-level name, **signature required**, **aliases not stages**.
- Works in the notebook but not in serving: check pip_requirements.

ON THE EXAM

Next: module 24 serves models: Foundation Model APIs, throughput modes and batch inference with ai_query.